

15 Days of Python — Control Flow Edition

A day-by-day practice challenge for if/else, loops, and everything that makes code make decisions

Curated by Erukulla Mahanth Arya

I put this together after realizing that control flow is the one topic people skim past too fast, thinking it's "just if-else and loops." It's not. It's the actual skeleton of how a program thinks. So instead of one long boring tutorial, here's 15 small, focused days — about 20–30 minutes each — that build on each other. No fluff, no giant projects, just enough reps to make if statements and loops feel automatic. Grab a notebook, open your editor, and let's go.

How to use this

Do one day at a time, in order — don't skip ahead even if a day looks easy, since later days quietly reuse earlier ideas. Type the code yourself instead of copy-pasting; typos are where the actual learning happens. And try to predict the output before you run anything, that's the habit that separates people who understand loops from people who just memorize syntax.

The 15 Days

Day 1 — If, Elif, Else, the honest way

Write a program that takes a number and prints whether it's positive, negative, or zero. Then extend it to grade a score out of 100 into A/B/C/F using elif chains. Focus on getting the order of conditions right — one wrong order and everything above it silently steals the answer.

Day 2 — Nested conditionals

Build a simple login check: username must match AND password must match, but give a different message depending on which one failed. This is your first real taste of nested if blocks, and why flattening them with 'and' is often cleaner.

Day 3 — The humble while loop

Print numbers 1 to 10 using while instead of for. Then write a guess-the-number game that keeps asking until the user gets it right. The goal here is to feel the danger of an infinite loop firsthand, and learn to always ask: what actually stops this loop?

Day 4 — For loops and range()

Loop through range(1, 21) and print only even numbers. Then print a multiplication table for any number the user enters. range() feels small today, but you'll use it every single day after this.

Day 5 — break and continue

Search a list of numbers for the first value greater than 50 and stop immediately using break. Then loop through 1–20 but skip every multiple of 3 using continue. Two tiny keywords, huge control.

Day 6 — Loops over strings

Count vowels in a sentence using a for loop. Then reverse a string manually (no slicing tricks allowed) just to prove to yourself that a string is really just a loopable sequence.

Day 7 — Nested loops

Print a 5x5 grid of stars using two nested for loops. Then build a simple pattern (a triangle of numbers works well). Nested loops trip people up mainly because of the inner/outer confusion — slow down and trace it by hand once.

Day 8 — Loops + conditionals together

Go through a list of exam scores and print PASS or FAIL for each, then count and print how many passed overall. This is the day it stops feeling like separate topics and starts feeling like one skill.

Day 9 — The else clause on loops

Search for a target number in a list using a for loop, and use the loop's own 'else' block to print 'not found' only if break was never hit. Most people don't know Python loops can do this — it's a small but genuinely useful trick.

Day 10 — List comprehensions as compressed control flow

Rewrite three of your earlier loops (evens, vowels, pass/fail) as one-line list comprehensions. You're not learning new logic here, just a tighter way to write control flow you already understand.

Day 11 — match-case (structural pattern matching)

Build a simple traffic-light responder: red/yellow/green each print a different action, plus a default case for anything unexpected. Think of match-case as elif's cleaner cousin for fixed sets of values.

Day 12 — Ternary expressions

Rewrite five of your earlier if/else blocks as one-line conditional expressions (value_if_true if condition else value_if_false). Great for short decisions, but resist the urge to force every if statement into this shape.

Day 13 — Error-aware control flow

Wrap a division calculator in try/except so it handles both division-by-zero and invalid input gracefully instead of crashing. try/except is control flow too — it decides what happens when things go wrong.

Day 14 — A mini FizzBuzz, but yours

Solve classic FizzBuzz first (1 to 100, multiples of 3 print Fizz, multiples of 5 print Buzz, both print FizzBuzz). Then invent your own twist rule and add it. If Day 14 feels easy, that's the whole point — it means the last 13 days worked.

Day 15 — Put it all together

Build a small number-guessing game with limited attempts, input validation, a win/lose message, and a play-again loop. Every tool from this challenge shows up here: while, if/elif/else, break, and try/except working as one program instead of fifteen separate exercises.

A quiet truth: control flow is basically decision-making and repetition, and once those two feel natural, most of programming stops feeling like memorizing syntax and starts feeling like just... describing what you want to happen. That shift is what this challenge is really for.

Before you close this PDF

Keep a small log of which day gave you the most trouble. That list is your actual weak-spot map, and it's worth revisiting a week after finishing all 15 days. Consistency beats intensity — 20 focused minutes a day will beat one exhausted 6-hour weekend every time.

— Erukulla Mahanth Arya

Quick Reference — Control Flow Cheat Sheet

Concept	When to reach for it
if / elif / else	Branch based on a condition; use elif chains instead of stacked ifs when only one branch should run.
while	Repeat while a condition holds and you don't know the exact number of repeats in advance.
for + range()	Repeat a known number of times, or step through a sequence.
break	Exit a loop immediately once you've found what you needed.
continue	Skip the rest of this iteration and move to the next one.
for...else	Run code only if the loop finished without hitting break.
match-case	Compare one value against several fixed possibilities, cleaner than a long elif chain.
Ternary expression	Pick between two short values in a single line.
try / except	Handle an error path as its own branch of control flow instead of letting it crash the program.

Progress Tracker

Days 1-5: ☐☐☐☐☐ Days 6-10: ☐☐☐☐☐ Days 11-15: ☐☐☐☐☐

Tick a box as you finish each day. If you fall behind, don't restart the whole challenge — just pick up from where you stopped. The point is the streak of understanding, not a perfect calendar.